Document number: P1324R1
Date: 2023-1-05
Audience: EWG
Reply-To: Mihail Naydenov <mihailnajdenov at gmail dot com>

RE: Yet another approach for constrained declarations

1 Revisions

R1

- Updated in the context of Reconsidering concepts in-place syntax (P2677)
- Added comparison to other languages
- Added “Details” section
- Removed discussion, related to out-of-date proposals.

R0 (2018)

Initial release, not presented.

2 Abstract

This proposal is direct continuation (a “reply”) to Yet another approach for constrained declarations (P1141).
It proposes to go “one step further” and allow specifying a type in the place of `auto` in all constrained declarations, suggested by P1141. In other words `void sort(Sortable auto& c);` becomes `void sort(Sortable S& c);` and as a result the specified typename `S` is introduced for the current function.

3 Summary

This proposal is a pure extension of P1141.
In a nutshell, allow

```
void f(Sortable auto x);
Sortable auto f();
Sortable auto x = f();
template <Sortable auto N> void f();
```

to also be written as:

```
void f(Sortable S x);
Sortable S f();
Sortable S x = f();
template <Sortable S N> void f();
```

Where `S` will be equivalent to a call to `decltype`, given the original P1141 declaration, or using the standard syntax:

Current

```
void f(Sortable auto&& x)
{
  using S = decltype(x); //<
        introduce S
  // use S
}

// OR, using the standard template
        syntax

template<Sortable S> //< introduce S
void f(S&& x)
{
  // use S
}
```

Proposed

```
void f(Sortable S&& x) //< introduce
        S
{
  // use S
}
```

§ 4 Motivation

The main motivation is explored in many other papers, like for example Concepts in-place syntax (P0745R0).
It basically boils down to the fact, it is often just too convenient to have the type be known as it makes the already verbose generic code slightly less so:

```
auto f(Concept auto&& x) { return something(std::forward<decltype(x)>
        (x)); } //< current
auto f(Concept T&& x) { return something(std::forward<T>(x)); } //<
        **proposed**

template<Number N> void f(N x, N y) { } //< current
void f(Number N x, N y) { }` //< **proposed**
```

Also, when we need to constrain multiple arguments, terse form becomes impractical

```
void f(Number auto a, Concept auto b) requires
        additional_constraint<decltype(a), decltype(b)> //< current
template<Number N, Concept C> void f(N a, C b) requires
        additional_constraint<N, C> //< current
```

```
void f(Number N a, Concept C b) requires additional_constraint<N, C> //<
        **proposed**
```

And sometimes it is even impossible to create a declaration using the terse form alone:

```
Concept auto f(Number auto a, AnotherConcept</*?decltype(return)?*/> auto
        b);  //< impossible
Concept R f(Number N a, AnotherConcept<R> U b); //< **proposed**
```

§ 4.1 Why this syntax

§ 4.1.1 Preserving established declaration rules

Consider what an `int n` states, reading it *right to left*.

"An object named 'n' of type integer."

But what is a type here? A type is representation in memory - technically - but a *constraint* semantically. In the case here the value can only be in certain range and only whole numbers. One can say, the value of a variable is constrained by its type.
In a way, we have been writing constrains for decades, constrains on values. Let's rephrase the above with that in mind:

"Object named 'n', that can have [only] values, belonging to the set of the 'int' type."

Ok, given the above, why should *constrains on types* be expressed any different?

Consider now `Number Num n`:

"Object named 'n', that can have [only] values, belonging to the set named 'Num', 'Num' can [only] be a type, belonging to the set of the 'Number' concept."

As You can see, we are not inventing anything, we are just_ recursively applying a constraint,_ right to left, as we always have!
In the case of constrained `auto`, we are simply *omitting* the name, effectively stating, we don't needed it - `Number auto n`.

> Sidenote, there are proposals, advertising the possibility to be able to omit the variable name also. Using the syntax from P1110R0, we can have fully unnamed, fully unconstrained declaration - `auto __` - alongside the fully named, fully constrained declaration we just described, *or* any combination b/w named/unnamed variable, named/unnamed type, constrained/unconstrained type and constrained/unconstrained variable.

§ 4.1.2 Preserving established relationship b/w `auto` and a type name

It must be pointed out, `auto` already stands-in for an omitted type! At the moment in all, but one case (structured bindings), if we replace `auto` with a type we end up with a valid

declaration, the meaning might change, but the declaration will be valid. This means the user already associates `auto` with a “type stand-in”, *even if* the declaration meaning changes by the substituting one with the other.
There is no reason not to continue this relationship - if the user sees an `auto`, he *should* be able to write a type name in its place. What this name represents will be new (a name introduction), but the context is also new; besides we don’t stride *too* much away from the established usage.

§ 4.1.3 It is actually the tersest form possible *ever*!

```
auto copy(InputIterator auto begin, decltype(begin) end, OutputIterator
        auto out) -> decltype(out);
template<InputIterator It, OutputIterator OIt> copy(It begin, It end, OIt
        out) -> OIt;
auto copy(InputIterator It begin, It end, OutputIterator OIt out) -> OIt;
        //< **proposed**

// Even compared to the original, concepts-instead-of-types syntax!
auto copy(InputIterator being, decltype(begin) end, OutputIterator out) -
        > decltype(out);
```

Compared to other contemporary languages, this syntax holds its ground. This is because we have **zero extra syntactical noise.**

Rust

```
notify(item: &impl Summary)
```

Proposed

```
notify(Summary S& item)
```

Carbon

```
PrintIt[T:! Printable](p: T*)
```

Proposed

```
PrintIt(Printable T* p)
```

Swift

```
someFunction<T: SomeThing>(someT: T)
```

Proposed

```
someFunction(SomeThing T someT)
```

C++ has long been considered verbose, when it comes to metaprogramming. Now the tables have turned and it is actually the cleanest.
In fact **the cleanest possible.**

> One can argue, if we had concepts-instead-of-types, *then* we would have “the cleanest possible” form. This is true as long as there are no extra constrains that need to be applied. In that case, a `decltype` would be needed and the results will not be as pretty.

§ 4.1.4 “OK, but still two declarations in one expression, this is madness!”

Type and variable declaration have always been in C++, inherited from C. And they still have their uses.

```
void process(class C& c);
//...
void something()
{
  process(get_c());
}
// ...
class C {...};
// ...
void process(C& c)
{
  // definition
}
```

also

```
class C
{
  // ...
private:
  const class Helper* sos() const;
};
```

As shown, two-in-one declaration is nothing new and although the above expressions can be written separately, this does not change the fact, an extremely similar syntax, with de facto the same meaning (introduce a typename inline) is already here - we just reinvent it for the new era.

> In contrast to the elaborate type specifier the declaration is limited to the current function declaration, as-if the type was declared as template parameter!

By this point it should be evident, enabling type name introduction in the proposed way, not only does not introduce new constructs and concepts, but *serve to reinforce established ones.*

§ 5 Details

In order to prevent confusion in the case, parameter names are omitted from a function declaration, **proposed** is to always treat function signatures, with two identifiers for a parameter, as the well known typename + param name pair.

```
void f(Number N); //< type and param name
```

In case `Number` is a **concept**, *this will fail to compile.*
The user will have to use a different syntax:

```
void f(Number auto); //< Not introdusing a typename
void f(Number N __); //< Using a placeholder syntax (P1110)

// (or any of the complete syntax options)
```

With the above limitation, generic functions will remain recognizable at a glance, even when using the terse syntax:

```
void f(Foo)       //< one id: standard function
void f(class Foo) //< one id and class: standard function
void f(Foo auto)  //< one id and auto: generic function

void f(Foo V)     //< two ids: standard function (as always)
void f(Foo F V)   //< *three* ids: generic function (proposed)
```

§ 6 Not proposed, but worth mentioning

Having return type name introduction, *combined* with trailing return type could enable us to have an interesting usage:

```
Number Ret  //< introduce Ret typename
function(Number N a, N b) -> decltype(something(x, b))
{
  Ret ret; //< easily declare and use the ret variable
  ...
  return ret;
}
```

also

```
template<class T>
Container Ret //< introduce Ret typename
function() -> std::map<Key, T, [](const T& a, const T& b){ ... },
        MyAllocator>
{
  Ret ret; //< easily declare and use the ret variable
  ...
  return ret;
}
```

Pretty neat.
Current alternatives are all inferior:

```
template<class T>
Container auto
function() -> std::map<Key, T, [](const T& a, const T& b){ ... },
        MyAllocator>
{
  std::map<Key, T, [](const T& a, const T& b){ ... }, MyAllocator> ret;
        //< duplication
  ...
```

```
  return ret;
}
```

or

```
template<class T>
Container auto
function() //< Return type, hidden, not part of the signature
{
  std::map<Key, T, [](const T& a, const T& b){ ... }, MyAllocator> ret;
  ...
  return ret;
}
```

7 Related work

P2677[1] has similar if not the same goals to current proposal. Here is side by side comparison:

Current

```
template<integral T>
T gcd(T l, T r);

template<typename X,
        typename ...Ts>
void wrap(X x, Ts&&
        ...ts);

template<Animal
        Predator,
        Animal Prey>
void
simulate(Predator
        &predator, Prey
        &prey);

template<typename T,
        typename U>
  requires
        convertible_to<T,
        U>
void f(T t, U u);

arithmetic auto a =
        calculation();
using A = decltype(a);
A a2 = refine(a);
```

P2677

```
// No more semantic
        change
auto gcd(integral
        auto:T l, T r)
        -> T;

// Forwarding only
        needs typenames
        for ts
void wrap(auto x,
        auto:Ts&&
        ...ts);

void
simulate(Animal
        auto:Predator
        &predator,
         Animal
        auto:Prey
        &prey);

void f(auto:T t, auto:U
        u)
  requires
        convertible_to<T,
        U>;


arithmetic auto:A a =
        calculation();
A a2 = refine(a);
```

This Proposal

```
// No more semantic
        change
auto gcd(integral T l,
        T r) -> T;

// Forwarding only
        needs typenames
        for ts
void wrap(auto x, Any
        Ts&& ...ts);


void
simulate(Animal
        Predator
        &predator,
         Animal Prey
        &prey);

void f(Any T t, Any U
        u)
  requires
        convertible_to<T,
        U>;


arithmetic A a =
        calculation();
A a2 = refine(a);
```

As you can see, both solutions strive to achieve similar goals. There are few differences. First, P2677 is considerably more verbose. It also relies on introducing a new variable introduction syntax and a new use of `auto`. In contrast, current solution is a natural extension of the language (see **Why this syntax**).
Second, P2677 has the ability to introduce names for unconstrained types. This proposal does not consider this a desirable feature. Unconstrained types are at this point *legacy*. All new types should be constrained. If the user really wants an unconstrained type, he/she is free to use a pass-all concept, like the above suggested `Any`, and *be explicit about it*. Later one can trivially track and replace the `Any` concept with a more constrained one, lessening the loose typing if desired.

§ 8 Conclusion

Allowing concepts to introduce the type they constrain will radically change the the way we write generic code. Doing it so with effectively **zero new constructs, reusing**

established practices, will ease the adoption of the terse form as the preferred one, possibly even for *unconstrained cases.*

---

1. Reconsidering concepts in-place syntax: Reconsidering concepts in-place syntax (P2677)↩